

ZÁPADOČESKÁ UNIVERZITA V PLZNI

Fakulta aplikovaných věd

Předmět KIV/ZPG

semestrální práce na téma:

Jednopatrové bludiště tvořené pravoúhlou sítí kostek

Autor: Pavel Kratochvíle

Datum: 3.04.2025

Email: pavelkr@gapps.zcu.cz

1 Souhlas s publikací

Souhlasím s vystavením této semestrální práce na stránkách katedry informatiky a výpočetní techniky a jejímu využití pro prezentaci pracoviště.

2 Implementace kolizí

2.1 Vektor pohybu

Aby se hráč mohl pohybovat konstantní rychlostí a zároveň byly skládány jeho vektory pohybu, implementoval jsem metodu `GetMovingVector`, která tuto funkcionalitu zajišťuje a vrací jednotkový vektor směru pohybu.

2.2 Reakce na pohyb

K reakcím na pohyb slouží metoda `MakeMove`, která přebírá parametr `double deltaTime`, což je časový rozdíl mezi jednotlivými snímky. Poté, podle příslušného stavu `KeyboardState.IsKeyDown`, zavolá metodu `Move` nad instancí hráče. Do této metody je vždy předán vektor pohybu, který závisí jak na `deltaTime`, tak na vektorech pohybu z předchozí metody. Dále také přebírá seznam modelů stěn a dveří k jednoznačnému určení kolizí.

2.3 Uskutečnění pohybu

Samotnou logiku kontroly kolize jsem implementoval následujícím způsobem. Aby se kolize nemusela kontrolovat s každým blokem, metoda `GetCubesInRadius` vždy vrátí seznam indexů všech kostek v určitém radiusu. Následně nad těmito kandidáty probíhají kontroly v obou směrech, zda jsou kroky `dx` realizovatelné. To se provádí kontrolou potenciální pozice hráče vzhledem ke kostkám v daném okruhu, a to pomocí jejich maximálních a minimálních hodnot `x/z`. Výsledek pohybu v každém směru se uloží do pomocných proměnných a na základě jejich kombinací se pohyb vykoná. Tato implementace zaručuje, že se hráč může klouzat po stěně, kdykoliv do ní narazí.

3 Implementace dodatečných úloh

1. Teleport (5 b) Teleportace z místa na místo. Implementace efektu pro hráče: přechod obrazu do bílé/z bílé do cílové scény.
2. Dveře a tajné chodby (5 b) Animace otvírání + interakce s hráčem (otevřeno/zavřeno/mění stav). Postačuje animace na úrovni Wolfenstein 3D.
3. Vícepatrové bludiště (15 b) Doplnění podpory pro více pater včetně kolize. Patra definována jako matice $n \times m \times k$. Implementace musí umožňovat vytvořit vícepatrové volné prostory a průlezy, přes které je možné nahlížet

do dalších pater. V bludišti se musí nacházet místa kde je možno plynule propadnout do nižšího patra.

3.1 Vícepatrové bludiště

Nejprve bylo vhodné začít nejkompexnější úlohou, jelikož její implementace vedla ke strukturální změně v kódu, která následně ovlivnila všechna další rozšíření. Ve zdrojovém kódu jsem si definoval novou třídu `Floor`, která obsahuje všechny objekty konkrétního patra. V hlavní třídě je pak seznam těchto pater. Každé patro má svou výšku a hloubku vztaženou k prvnímu patru bludiště.

Kolize jsou řešeny tak, že program kontroluje pouze objekty v patře, ve kterém se aktuálně nacházíme. Existují také místa, kde lze propadnout mezi patry. Tato místa jsou v reprezentaci označena jako propad podlahou (`>`) a propad stropem (`<`). Pokud chceme vytvořit průchod mezi patry, je nutné na stejnou souřadnici ve dvou sousedících patrech umístit příslušné propady (nahoru a dolů).

Gravitace je řešena jednoduše pomocí základní fyziky. Kontroluje se, zda se kamera nachází na zemi. Pokud ne, k vektoru rychlosti se přičítá součin gravitační konstanty a časového rozdílu mezi snímky. Pád se ukončí v momentě, kdy kamera dosáhne požadované výšky, což je výška patra násobená jeho hloubkou.

Vzhledem k tomu, že už byla implementována základní fyzika, využil jsem ji i pro skákání v bludišti. To je realizováno tak, že při stisku mezerníku se zvýší Y-složka vektoru rychlosti, čímž se simuluje skok.

3.2 Pohybující se dveře

Otevírání a zavírání dveří je velmi jednoduché. Pokud se hráč nachází alespoň 3 metry od středu dveří, může je otevřít stisknutím klávesy `E`. Tímto tlačítkem lze dveře také opět zavřít.

Logika otevírání a zavírání funguje následovně: Při stisku tlačítka program iteruje přes všechny dveře v aktuálním patře a kontroluje, zda se hráč nachází dostatečně blízko. Pokud ano, ověří se, zda dané dveře nemají proměnnou `isOpening` nastavenou na `true` a zda ještě nejsou otevřené. Pokud tyto podmínky jsou splněny, metoda `doorMove` provádí posun dveří při každém snímku – na základě `deltaTime` a `doorSpeed` směrem k cílové pozici.

Směr, kterým se dveře otevírají, určuje metoda `GetNearestWall`. Ta zjišťuje, zda se v bezprostřední blízkosti dveří nachází zeď, směrem ke které by se dveře mohly otevřít.

Podobná logika platí i při zavírání dveří – místo posunu na cílovou pozici se dveře vracejí zpět do své výchozí (defaultní) pozice, tedy tam, kde byly původně umístěny.

Podobnou implementaci mají i tajné dveře. Ty jsou vizuálně vykresleny stejně jako běžné bloky, ale zároveň se chovají jako dveře a lze je otevírat. V textové reprezentaci mapy jsou označeny písmenem `G`.

3.3 Teleport

Teleport je na mapě reprezentován jako tyrkysový altánek. Pokud hráč vstoupí do jeho hitboxu, začne se odpočítávat dvousekundové zpoždění, během kterého se uživateli začne zesvětlovat obrazovka. Toto vizuální zobrazení je realizováno pomocí shaderu, který přebírá hodnotu typu `float` představující uplynulý čas v milisekundách. Po dokončení teleportace probíhá opačný přechod – obraz se plynule mění z bílé zpět do barevného zobrazení herního světa.

Po teleportaci se hráč objeví na náhodném teleportu v náhodném patře. Jedná se tedy o jediný způsob, jak se v rámci hry pohybovat do vyšších pater. Nakonec se také aktualizuje hodnota aktuálního patra, do kterého byl hráč teleportován, aby správně fungovaly kolizní detekce se stěnami a další logika spojená s daným patrem.

4 Spuštění a ovládání

Aplikaci lze spustit dvěma způsoby. Za prvé se můžeme přesunout z kořenového adresáře do podadresáře `src`, následně do složky `ConsoleApp1` a zde pomocí příkazové řádky aplikaci spustit příkazem `dotnet run`. Druhou možností je využít soubor `.exe` v adresáři `bin`, který lze rovněž spustit přes příkazovou řádku.

Pokud bychom chtěli spustit hru s vlastní mapou a více patry, stačí mapy jednoduše uložit jako soubor s příponou `.txt` do složky `Maps` a jako parametry příkazové řádky uvést názvy těchto souborů. Pokud žádné mapy ne zadáme, aplikace se spustí s výchozí mapou.

příklad: `dotnet run map1.txt map2.txt map3.txt`

Dveře jsou ve videohře odlišeny šedou barvou. Průchod dveřmi je velmi jednoduchý – stačí se nacházet v jejich dosahu do dvou metrů a stisknutím tlačítka `E` dveře otevřeme. Lze je také zpětně zavřít stisknutím tlačítka `Q`. Doporučuji **třípatrové bludiště**, které je ve výchozím nastavení, nebo větší **dvoupatrové bludiště** načtené pomocí parametrů příkazové řádky: `map1.txt` `map3.txt`. Pokud by uživatel chtěl **jednopatrové bludiště** nebo bludiště s **více patry**, je nutné správně upravit zápisy v *textové reprezentaci*.